Project Report

DSA Project

Smart Parking Management System

Course Code : PETV156

Course Title: The complete DSA Roadmap

Submitted By:

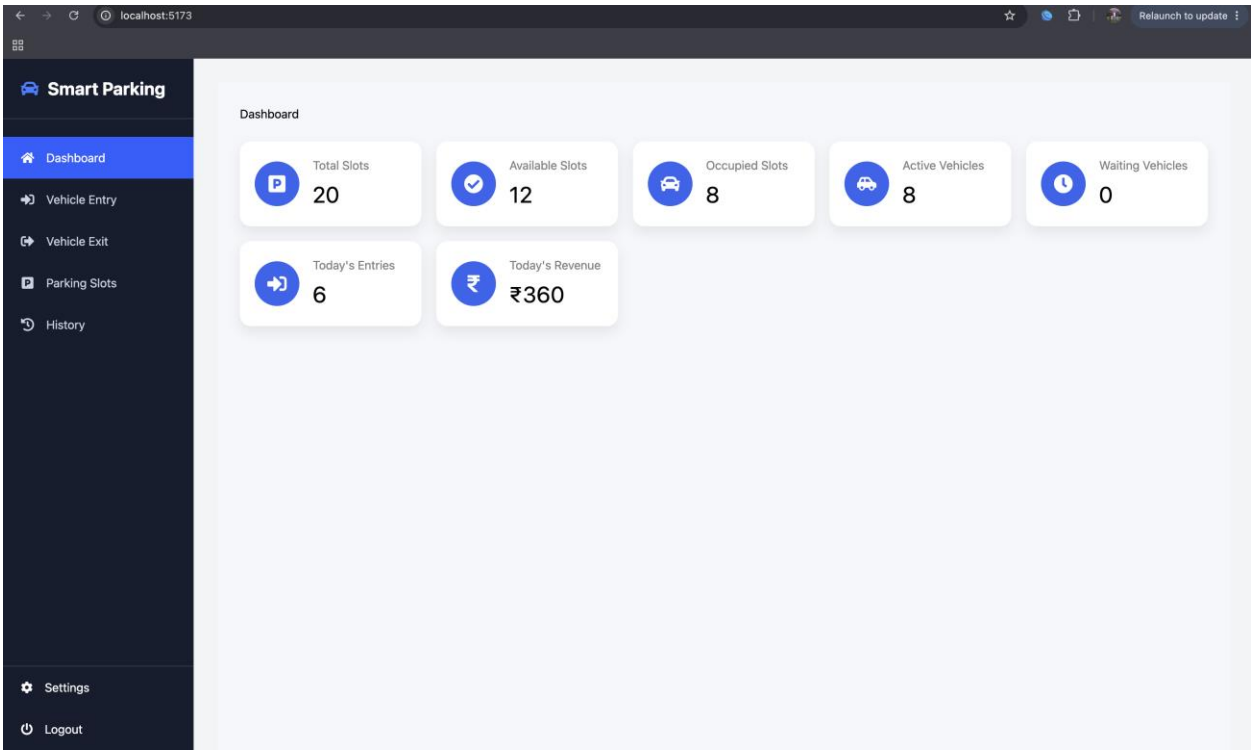
Name – Vikash Kumar

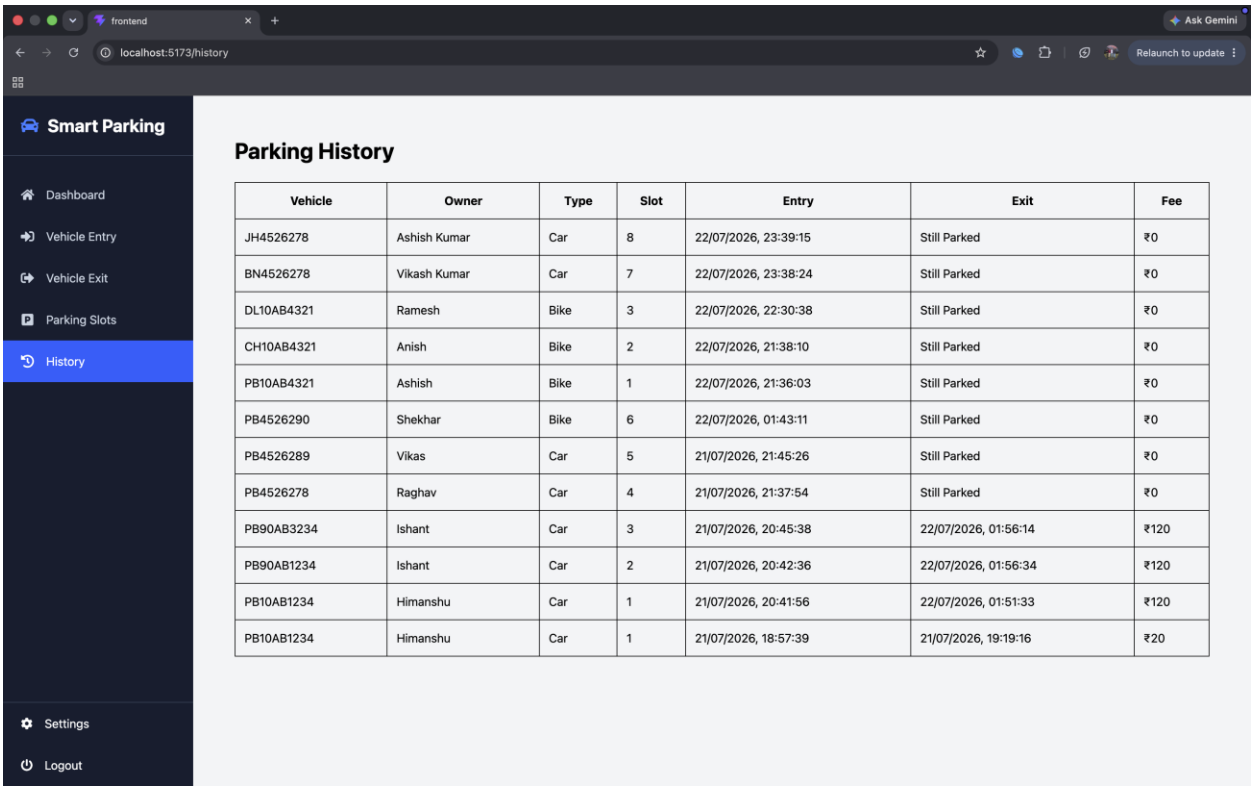
Reg No. – 12311725

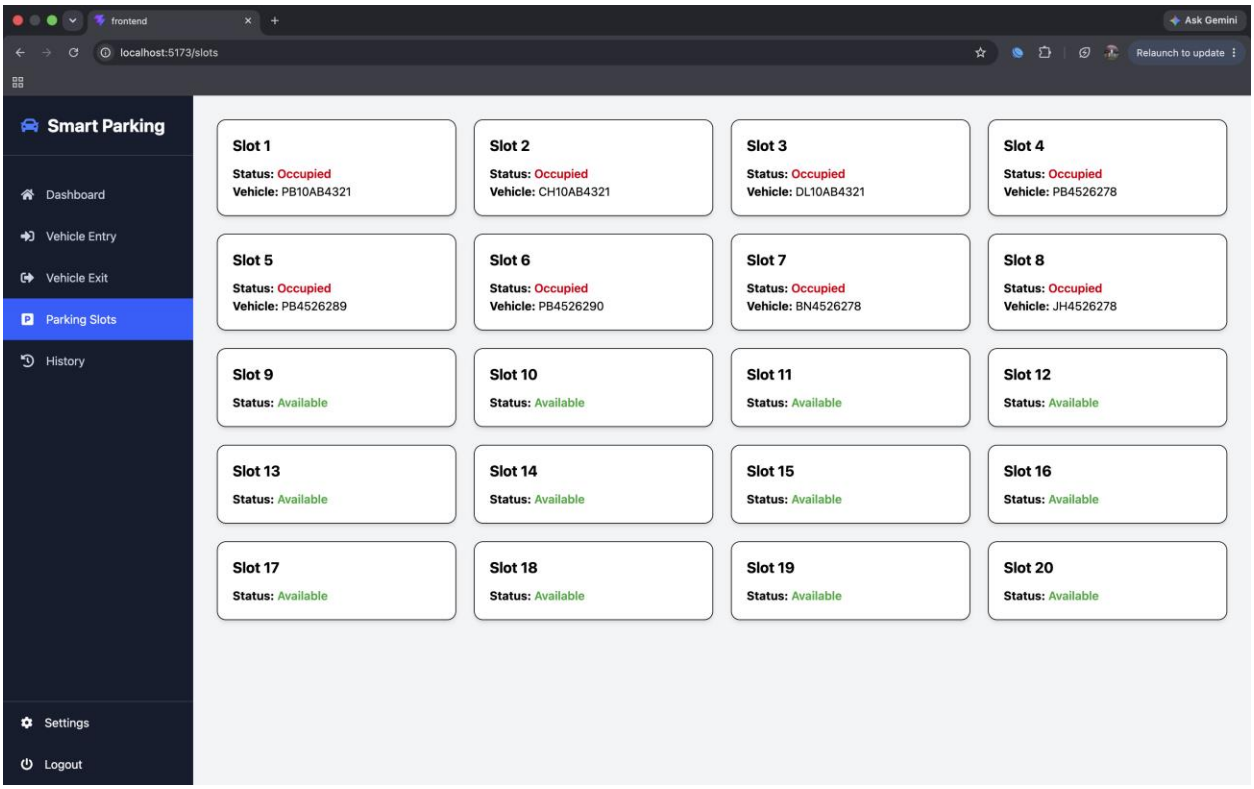
Submitted To:

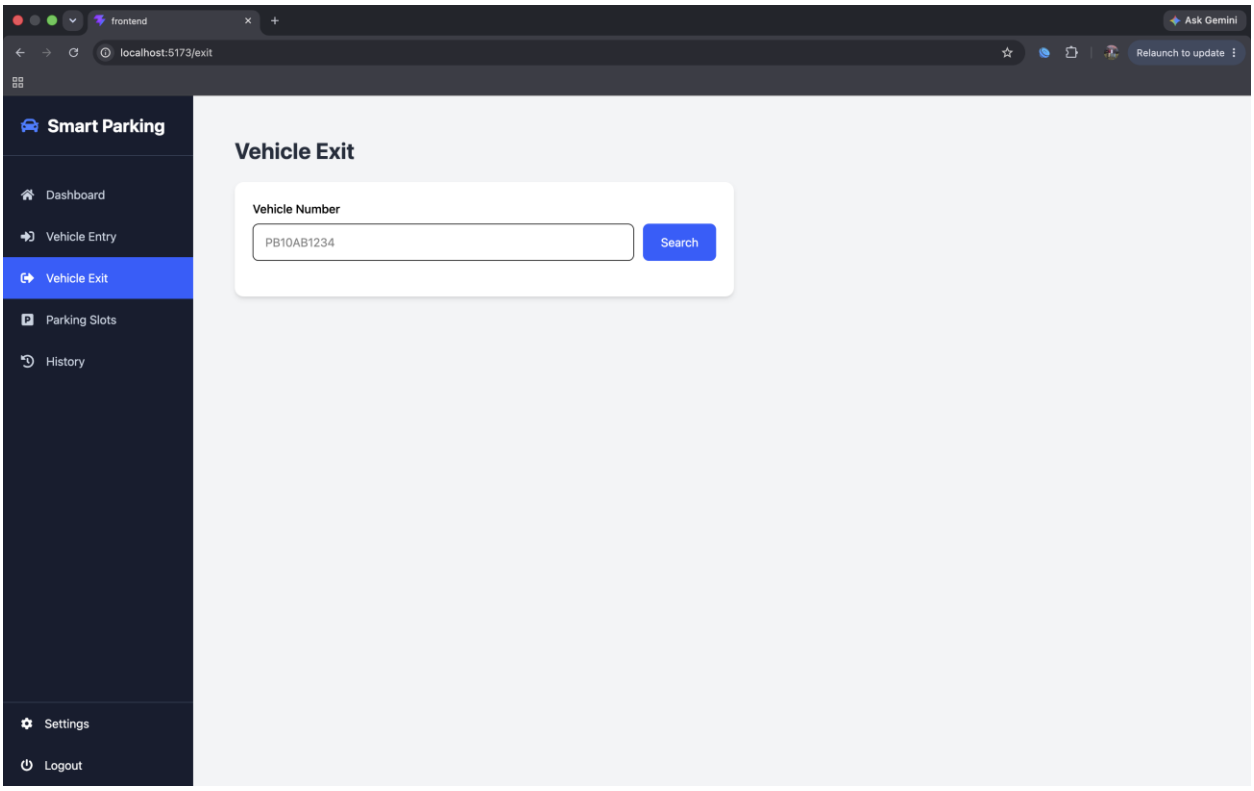
Deepak Kumar

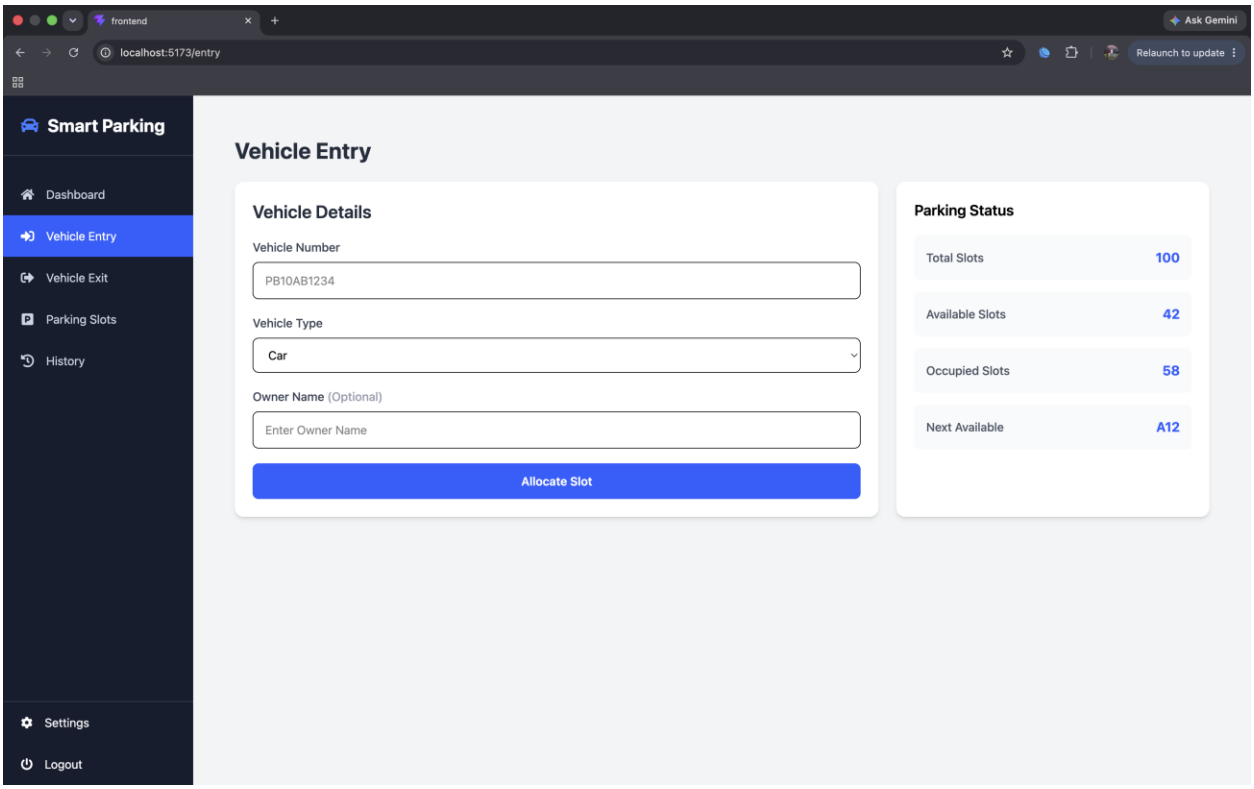
1. Front Page


localhost:5173
Relaunch to update
Smart Parking
Dashboard
Vehicle Entry
Vehicle Exit
Parking Slots
History
Settings
Logout
Dashboard
Total Slots
20
Available Slots
12
Occupied Slots
8
Active Vehicles
8
Waiting Vehicles
0
Today's Entries
6
Today's Revenue
₹360


frontend
Ask Gemini
localhost:5173/history
Relaunch to update
Smart Parking
Dashboard
Vehicle Entry
Vehicle Exit
Parking Slots
History
Settings
Logout
Parking History
Vehicle Owner Type Slot Entry Exit Fee
JH4526278 Ashish Kumar Car 8 22/07/2026, 23:39:15 Still Parked ₹0
BN4526278 Vikash Kumar Car 7 22/07/2026, 23:38:24 Still Parked ₹0
DL10AB4321 Ramesh Bike 3 22/07/2026, 22:30:38 Still Parked ₹0
CH10AB4321 Anish Bike 2 22/07/2026, 21:38:10 Still Parked ₹0
PB10AB4321 Ashish Bike 1 22/07/2026, 21:36:03 Still Parked ₹0
PB4526290 Shekhar Bike 6 22/07/2026, 01:43:11 Still Parked ₹0
PB4526289 Vikas Car 5 21/07/2026, 21:45:26 Still Parked ₹0
PB4526278 Raghav Car 4 21/07/2026, 21:37:54 Still Parked ₹0
PB90AB3234 Ishant Car 3 21/07/2026, 20:45:38 22/07/2026, 01:56:14 ₹120
PB90AB1234 Ishant Car 2 21/07/2026, 20:42:36 22/07/2026, 01:56:34 ₹120
PB10AB1234 Himanshu Car 1 21/07/2026, 20:41:56 22/07/2026, 01:51:33 ₹120
PB10AB1234 Himanshu Car 1 21/07/2026, 18:57:39 21/07/2026, 19:19:16 ₹20


frontend
Ask Gemini
localhost:5173/slots
Relaunch to update
Smart Parking
Dashboard
Vehicle Entry
Vehicle Exit
Parking Slots
History
Settings
Logout
Slot 1
Status: Occupied
Vehicle: PB10AB4321
Slot 2
Status: Occupied
Vehicle: CH10AB4321
Slot 3
Status: Occupied
Vehicle: DL10AB4321
Slot 4
Status: Occupied
Vehicle: PB4526278
Slot 5
Status: Occupied
Vehicle: PB4526289
Slot 6
Status: Occupied
Vehicle: PB4526290
Slot 7
Status: Occupied
Vehicle: BN4526278
Slot 8
Status: Occupied
Vehicle: JH4526278
Slot 9
Status: Available
Slot 10
Status: Available
Slot 11
Status: Available
Slot 12
Status: Available
Slot 13
Status: Available
Slot 14
Status: Available
Slot 15
Status: Available
Slot 16
Status: Available
Slot 17
Status: Available
Slot 18
Status: Available
Slot 19
Status: Available
Slot 20
Status: Available


frontend
Ask Gemini
localhost:5173/exit
Relaunch to update
Smart Parking
Dashboard
Vehicle Entry
Vehicle Exit
Parking Slots
History
Settings
Logout
Vehicle Exit
Vehicle Number
PB10AB1234
Search



Deployed Link : https://smart-parking-manager-frontend-ny98.onrender.com/

2. ABSTRACT

The **Smart Parking Management System** is a web-based application developed to automate and simplify the management of parking spaces in an organized and efficient manner. Traditional parking systems often rely on manual processes, which lead to increased waiting time, inefficient utilization of parking slots, and difficulties in managing vehicle records. This project addresses these challenges by providing an automated solution that manages vehicle entry, exit, parking slot allocation, waiting queues, parking history, and fee calculation.

The application is developed using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)** for the frontend and backend development. A significant feature of this project is the integration of **C++**, which is used to implement Data Structures and Algorithms (DSA) for parking slot allocation. The parking allocation logic is executed using a C++ program that communicates with the Node.js backend through child processes.

The system maintains real-time information about available parking slots and automatically assigns the nearest available slot to an incoming vehicle. If all parking slots are occupied, vehicles are added to a **FIFO (First In First Out) waiting queue**, ensuring fair allocation when a parking space becomes available. Once a vehicle exits, the first vehicle in the waiting queue is automatically allocated the newly available parking slot.

The system also maintains complete parking records, including vehicle details, entry time, exit time, parking duration, and parking fee. A dashboard provides real-time statistics such as total parking slots, occupied slots, available slots, active vehicles, waiting vehicles, daily entries, and today's revenue.

The project demonstrates the practical implementation of Data Structures and Algorithms in a real-world application while combining modern web technologies with efficient backend processing. It offers a scalable, user-friendly, and efficient solution for smart parking management.

Keywords: MERN Stack, Smart Parking System, Data Structures, Queue, C++, MongoDB, React.js, Node.js, Express.js, Parking Management.

3. INTRODUCTION

3.1 Introduction

Parking management has become one of the major challenges in modern cities due to the continuously increasing number of vehicles. Conventional parking systems require manual monitoring, resulting in traffic congestion, inefficient space utilization, and delays in parking allocation. These problems reduce user convenience and increase operational complexity.

To overcome these limitations, a **Smart Parking Management System** has been developed. The system automates the entire parking process, beginning with vehicle entry and ending with vehicle exit and fee calculation. It efficiently allocates parking slots, maintains parking records, and manages waiting vehicles whenever the parking area becomes full.

Unlike traditional CRUD-based web applications, this project integrates **Data Structures and Algorithms (DSA)** through C++ programming. The parking slot allocation algorithm is

implemented in C++ for better performance and to demonstrate algorithmic problem-solving. The Node.js backend executes the C++ program whenever a new vehicle enters the parking lot.

The project also introduces a **FIFO Queue** to handle situations where all parking slots are occupied. Instead of rejecting incoming vehicles, the system stores them in a waiting queue. As soon as a parking slot becomes vacant, the vehicle at the front of the queue is automatically assigned that slot.

The application is designed with a modern React.js frontend, Express.js backend, MongoDB database, and C++ DSA integration, making it a complete full-stack software engineering project.

3.2 Problem Statement

Traditional parking systems suffer from several limitations:

- Manual parking management.
- Difficulty in identifying available parking slots.
- Time-consuming vehicle tracking.
- No automatic waiting queue management.
- Inefficient utilization of parking resources.
- Manual parking fee calculation.
- Lack of centralized parking history.

These issues increase operational costs and reduce customer satisfaction.

3.3 Objectives

The primary objectives of this project are:

- To automate parking slot allocation.
- To reduce manual intervention in parking management.
- To maintain real-time parking slot availability.
- To implement Data Structures and Algorithms using C++.

- To maintain complete vehicle entry and exit records.
- To calculate parking fees automatically.
- To implement a FIFO Queue for waiting vehicles.
- To provide a dashboard with real-time parking statistics.

3.4 Scope of the Project

The Smart Parking Management System can be used in:

- Shopping malls
- Hospitals
- Educational institutions
- Airports
- Railway stations
- Office buildings
- Residential societies
- Commercial parking lots

The project can also be extended to integrate IoT sensors, RFID, QR Code scanning, ANPR (Automatic Number Plate Recognition), and online payment systems.

3.5 Technologies Used

Technology	Purpose
React.js	Frontend Development
Node.js	Backend Runtime
Express.js	REST API Development
MongoDB	Database Management
Mongoose	MongoDB ODM
C++	Parking Slot Allocation Algorithm
Queue (DSA)	Waiting Vehicle Management
Axios	API Communication
CSS	User Interface Styling

4. ALGORITHM

Algorithm 1: Vehicle Entry

1. User enters vehicle details.
2. Validate all input fields.
3. Check whether the vehicle already exists in the parking area.
4. Search for an available parking slot.
5. If a slot is available:
    a. Execute the C++ parking allocation program.
    b. Allocate the nearest available slot.
    c. Store the parking information in the database.
    d. Update the slot status to occupied.
6. If no slot is available:
    a. Add the vehicle to the waiting queue.
    b. Assign queue position.
7. Display the parking confirmation or waiting queue message.

Algorithm 2: Vehicle Exit

1. User enters vehicle number.
2. Search for the active parking record.
3. Calculate parking duration.
4. Calculate parking fee.
5. Mark parking slot as available.
6. Update parking history.
7. Check waiting queue.
8. If queue is not empty:
    a. Select first vehicle from queue (FIFO).
    b. Allocate newly available parking slot.
    c. Remove vehicle from queue.
    d. Update remaining queue positions.
9. Display exit confirmation.

Algorithm 3: Parking Fee Calculation

Input:
Entry Time
Exit Time

Duration = Exit Time − Entry Time

If duration < 1 hour
Fee = ₹20

Else
Fee = Duration × ₹20

Algorithm 4: Waiting Queue (FIFO)

If Parking Full

↓

Insert Vehicle at Rear

↓

Vehicle Waits

↓

Vehicle Exit Occurs

↓

Front Vehicle Gets Slot

↓

Remove Front Vehicle

↓

Update Queue Position

Data Structures Used

Data Structure	Purpose
Queue	Waiting vehicle management
Array	Parking slot processing
Hashing (MongoDB Indexes)	Fast vehicle lookup

5. RESULTS

The Smart Parking Management System was successfully developed and tested using the MERN Stack and C++. The application performs all major parking management operations efficiently.

Features Successfully Implemented

- Parking Slot Initialization
- Vehicle Entry
- Vehicle Exit
- Automatic Slot Allocation
- Parking Fee Calculation
- Parking History
- Waiting Queue Management
- Dashboard Analytics
- Real-time Parking Statistics
- MongoDB Integration
- REST API Communication
- C++ Algorithm Integration

Dashboard Output

The dashboard displays:

- Total Parking Slots
- Available Parking Slots
- Occupied Parking Slots
- Active Vehicles
- Waiting Vehicles
- Today's Entries
- Today's Revenue

Performance

The system successfully allocates parking slots automatically, updates parking availability in real time, manages waiting vehicles using FIFO Queue, and calculates parking fees accurately. The integration of C++ with the MERN Stack demonstrates efficient implementation of Data Structures and Algorithms in a practical software application.

6. CONCLUSION

The Smart Parking Management System successfully automates the complete parking process using modern web technologies and Data Structures. The project minimizes manual work, improves parking efficiency, and provides a user-friendly interface for managing parking operations.

The integration of React.js, Node.js, Express.js, MongoDB, and C++ creates a scalable and efficient solution capable of handling real-world parking scenarios. The implementation of Queue (FIFO) ensures fair parking allocation whenever the parking area becomes full, while the C++ parking allocation algorithm demonstrates practical usage of Data Structures and Algorithms.

Overall, the project fulfills all intended objectives, including automatic parking slot allocation, vehicle tracking, waiting queue management, parking fee calculation, and dashboard analytics. The developed system is reliable, efficient, and can be extended in

the future with technologies such as IoT sensors, RFID, online payments, and mobile applications.

7. REFERENCES

1. React.js Official Documentation – https://react.dev/
2. Node.js Official Documentation – https://nodejs.org/
3. Express.js Documentation – https://expressjs.com/
4. MongoDB Documentation – https://www.mongodb.com/docs/
5. Mongoose Documentation – https://mongoosejs.com/docs/
6. C++ Reference – https://en.cppreference.com/
7. MDN Web Docs – https://developer.mozilla.org/
8. Axios Documentation – https://axios-http.com/
9. Visual Studio Code Documentation – https://code.visualstudio.com/docs
10. GeeksforGeeks – https://www.geeksforgeeks.org/

8. Source Code

Smart-Parking-Management-System/

```
│
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   └── ...
│
├── backend/
│   ├── src/
```

```
│   │   ├── controllers/
│   │   ├── models/
│   │   ├── routes/
│   │   ├── config/
│   │   ├── services/
│   │   └── app.js
│   ├── cpp/
│   │   ├── parking/
│   │   │   ├── parking.cpp
│   │   │   └── parking
│   │   └── queue/
│   ├── package.json
│   └── .env.example
│
├── README.me
```

Backend Folder

Server.js

```
require("dotenv").config();
const app = require("./app");
const connectDB = require("./config/db");

const PORT = process.env.PORT || 5000;

connectDB();
```

```
app.listen(PORT, () => {
console.log(`Server running on ${PORT}`);
});
```

App.js

```
const express = require("express");
const cors = require("cors");
const morgan = require("morgan");

const parkingRoutes = require("./routes/parkingRoutes");
const vehicleRoutes = require("./routes/vehicleRoutes");
const dashboardRoutes = require("./routes/dashboardRoutes");
const queueRoutes = require("./routes/queueRoutes");



const app = express();

app.use(cors());
app.use(express.json());
app.use(morgan("dev"));

app.use("/api/slots", parkingRoutes);
app.use("/api/vehicles", vehicleRoutes);
```

```
app.use("/api/dashboard", dashboardRoutes);
app.use("/api/queue", queueRoutes);

app.get("/", (req, res) => {
res.json({
message: "Smart Parking API Running"
});
});

module.exports = app;
```

Package.json file

```
{
"name": "backend",
"version": "1.0.0",
"description": "",
"main": "index.js",
"scripts": {
"test": "echo \"Error: no test specified\" && exit 1",
"dev": "nodemon src/server.js"
},
"keywords": [],
"author": "",
"license": "ISC",
```

```
"type": "commonjs",
"dependencies": {
"cors": "^2.8.6",
"dotenv": "^17.4.2",
"express": "^5.2.1",
"mongodb": "^7.5.0",
"mongoose": "^9.8.0",
"morgan": "^1.11.0"
},
"devDependencies": {
"nodemon": "^3.1.14"
}
}
```

Then src folder

Config/db.js

```
const mongoose = require("mongoose");

const connectDB = async () => {
try {
await mongoose.connect(process.env.MONGO_URI);

console.log("MongoDB Connected");
} catch (err) {
```

```
console.log(err.message);
process.exit(1);
}
};

module.exports = connectDB;
```

Controllers

DashboardControllers.js

```
const Slot = require("../models/Slot");
const ParkingRecord = require("../models/ParkingRecord");
const WaitingVehicle = require("../models/WaitingVehicle");

const getDashboardData = async (req, res) => {
try {
const totalSlots = await Slot.countDocuments();

const availableSlots = await Slot.countDocuments({
isOccupied: false,
});

const occupiedSlots = await Slot.countDocuments({
isOccupied: true,
});
```

```
const activeVehicles = await ParkingRecord.countDocuments({
exitTime: null,
});
const waitingVehicles = await WaitingVehicle.countDocuments({
status: "Waiting",
});

const today = new Date();
today.setHours(0, 0, 0, 0);

const todayEntries = await ParkingRecord.countDocuments({
entryTime: {
$gte: today,
},
});

const todayRecords = await ParkingRecord.find({
exitTime: {
$gte: today,
},
});

const todayRevenue = todayRecords.reduce(
(sum, record) => sum + record.parkingFee,
```

```
0,
);

res.json({
totalSlots,
availableSlots,
occupiedSlots,
activeVehicles,
waitingVehicles,
todayEntries,
todayRevenue,
});
} catch (error) {
res.status(500).json({
message: error.message,
});
}
};

module.exports = {
getDashboardData,
};
```

ParkingController.js

```
const Slot = require("../models/Slot");

// Initialize parking slots
const initializeSlots = async (req, res) => {
try {
// Get totalSlots from request body
const { totalSlots } = req.body;

// Validate input
if (!totalSlots || totalSlots <= 0) {
return res.status(400).json({
success: false,
message: "Please enter a valid number of slots",
});
}
// Check if slots already exist
const existingSlots = await Slot.countDocuments();

if (existingSlots > 0) {
return res.status(400).json({
success: false,
message: "Parking slots are already initialized",
});
}
```

```
// Create an array of slots
const slots = [];

for (let i = 1; i <= totalSlots; i++) {
slots.push({
slotNumber: i,
});
}

// Save all slots to MongoDB
await Slot.insertMany(slots);

res.status(201).json({
success: true,
message: `${totalSlots} slots created successfully`,
});

} catch (error) {
res.status(500).json({
success: false,
message: error.message,
});
}
};
```

```
const getAllSlots = async (req, res) => {
try {
const slots = await Slot.find().sort({ slotNumber: 1 });

res.status(200).json({
success: true,
total: slots.length,
data: slots,
});
} catch (error) {
res.status(500).json({
success: false,
message: error.message,
});
}
};

const getAvailableSlots = async (req, res) => {
try {
const slots = await Slot.find({ isOccupied: false }).sort({
slotNumber: 1,
});
```

```
res.status(200).json({
success: true,
availableSlots: slots.length,
data: slots,
});
} catch (error) {
res.status(500).json({
success: false,
message: error.message,
});
}
};

const getSlotByNumber = async (req, res) => {
try {
const { slotNumber } = req.params;
const slotNumberInt = Number(slotNumber);

if (isNaN(slotNumber)) {
return res.status(400).json({
success: false,
message: "Invalid slot number",
});
}
```

```
const slot = await Slot.findOne({ slotNumber });

if (!slot) {
return res.status(404).json({
success: false,
message: "Slot not found",
});
}

res.status(200).json({
success: true,
data: slot,
});
} catch (error) {
res.status(500).json({
success: false,
message: error.message,
});
}
};



module.exports = {
```

```
initializeSlots,
getAllSlots,
getAvailableSlots,
getSlotByNumber,
};
```

QueueController.js

```
const queueService = require("../services/queueService");

const enqueue = async (req, res) => {
try {
const queue = await queueService.enqueueVehicle(req.body);

res.status(201).json({
success: true,
message: "Vehicle added to waiting queue.",
data: queue,
});
} catch (error) {
res.status(400).json({
success: false,
message: error.message,
});
}
```

```
};

const dequeue = async (req, res) => {
try {
const vehicle = await queueService.dequeueVehicle();

res.status(200).json({
success: true,
message: "Vehicle removed from waiting queue.",
data: vehicle,
});
} catch (error) {
res.status(400).json({
success: false,
message: error.message,
});
}
};

const getQueue = async (req, res) => {
try {
const queue = await queueService.getQueue();

res.status(200).json({
```

```
success: true,
count: queue.length,
data: queue,
});
} catch (error) {
res.status(500).json({
success: false,
message: error.message,
});
}
};


module.exports = {
enqueue,
dequeue,
getQueue,
};
```

VehicleController.js

```
const Vehicle = require("../models/Vehicle");
const Slot = require("../models/Slot");
const WaitingVehicle = require("../models/WaitingVehicle");
const ParkingRecord = require("../models/ParkingRecord");
```

```
const { execFile } = require("child_process");
const util = require("util");
const path = require("path");

const execFilePromise = util.promisify(execFile);

const cppPath = path.join(
process.cwd(),
"cpp",
"parking",
"parking"
);
console.log("C++ Path:", cppPath);

const vehicleEntry = async (req, res) => {
try {
const { vehicleNumber, ownerName, vehicleType } = req.body;

// Validate input
if (!vehicleNumber || !ownerName || !vehicleType) {
return res.status(400).json({
success: false,
message: "All fields are required",
});
```

```
}

// Check if vehicle is already parked
const parkedVehicle = await Slot.findOne({
vehicleNumber,
isOccupied: true,
});

if (parkedVehicle) {
return res.status(400).json({
success: false,
message: "Vehicle is already parked",
});
}

// Get all available slots
const freeSlots = await Slot.find({
isOccupied: false,
}).sort({ slotNumber: 1 });

// Parking Full → Add vehicle to waiting queue
if (freeSlots.length === 0) {
// Check if already waiting
const alreadyWaiting = await WaitingVehicle.findOne({
```

```
vehicleNumber,
status: "Waiting",
});

if (alreadyWaiting) {
return res.status(400).json({
success: false,
message: "Vehicle is already in the waiting queue.",
});
}

// Queue position
const queueLength = await WaitingVehicle.countDocuments({
status: "Waiting",
});

await WaitingVehicle.create({
vehicleNumber,
ownerName,
vehicleType,
entryTime: new Date(),
queuePosition: queueLength + 1,
status: "Waiting",
});
```

```
return res.status(200).json({
success: true,
message: "Parking Full. Vehicle added to waiting queue.",
queuePosition: queueLength + 1,
});
}

// Convert slot numbers into string for C++
const slotString = freeSlots
.map((slot) => slot.slotNumber)
.join(" ");

// Run C++ parking allocation
const { stdout } = await execFilePromise(cppPath, [slotString]);

const allocatedSlotNumber = parseInt(stdout.trim());

// Find allocated slot
const availableSlot = await Slot.findOne({
slotNumber: allocatedSlotNumber,
});

if (!availableSlot) {
return res.status(500).json({
```

```
success: false,
message: "C++ failed to allocate a valid slot.",
});
}

// Find or create vehicle
let vehicle = await Vehicle.findOne({ vehicleNumber });

if (!vehicle) {
vehicle = await Vehicle.create({
vehicleNumber,
ownerName,
vehicleType,
});
}

// Occupy slot
availableSlot.isOccupied = true;
availableSlot.vehicleNumber = vehicleNumber;

await availableSlot.save();

// Create parking history
const parkingRecord = await ParkingRecord.create({
vehicle: vehicle._id,
```

```
slot: availableSlot._id,
entryTime: new Date(),
});

return res.status(201).json({
success: true,
message: "Vehicle parked successfully",
data: {
slotNumber: availableSlot.slotNumber,
vehicleNumber,
ownerName,
vehicleType,
entryTime: parkingRecord.entryTime,
},
});

} catch (error) {
return res.status(500).json({
success: false,
message: error.message,
});
}
};
const vehicleExit = async (req, res) => {
```

```
try {
const { vehicleNumber } = req.body;

// Validate input
if (!vehicleNumber) {
return res.status(400).json({
success: false,
message: "Vehicle number is required",
});
}

// Find vehicle
const vehicle = await Vehicle.findOne({ vehicleNumber });

if (!vehicle) {
return res.status(404).json({
success: false,
message: "Vehicle not found",
});
}

// Find active parking record
const parkingRecord = await ParkingRecord.findOne({
vehicle: vehicle._id,
```

```
exitTime: null,
});

if (!parkingRecord) {
return res.status(404).json({
success: false,
message: "Vehicle is not currently parked",
});
}

// Find occupied slot
const slot = await Slot.findById(parkingRecord.slot);

if (!slot) {
return res.status(404).json({
success: false,
message: "Parking slot not found",
});
}

// Exit time
const exitTime = new Date();

// Calculate parking duration
const durationInMs = exitTime - parkingRecord.entryTime;
```

```
const durationInHours = Math.ceil(durationInMs / (1000 * 60 * 60));

// Fee calculation
let hourlyRate = 20;

if (vehicle.vehicleType.toLowerCase() === "bike") {
hourlyRate = 10;
}

const parkingFee = durationInHours * hourlyRate;

// Update parking record
parkingRecord.exitTime = exitTime;
parkingRecord.parkingFee = parkingFee;

await parkingRecord.save();

// Free parking slot
slot.isOccupied = false;
slot.vehicleNumber = null;

await slot.save();

// ================================
// FIFO Queue Logic Starts Here
```

```
// ================================

const waitingVehicle = await WaitingVehicle.findOne({
status: "Waiting",
}).sort({ queuePosition: 1 });

if (waitingVehicle) {
// Find or create vehicle
let nextVehicle = await Vehicle.findOne({
vehicleNumber: waitingVehicle.vehicleNumber,
});

if (!nextVehicle) {
nextVehicle = await Vehicle.create({
vehicleNumber: waitingVehicle.vehicleNumber,
ownerName: waitingVehicle.ownerName,
vehicleType: waitingVehicle.vehicleType,
});
}

// Assign freed slot
slot.isOccupied = true;
slot.vehicleNumber = waitingVehicle.vehicleNumber;

await slot.save();
```

```
// Create parking record
await ParkingRecord.create({
vehicle: nextVehicle._id,
slot: slot._id,
entryTime: new Date(),
});

// Remove from waiting queue
await WaitingVehicle.deleteOne({
_id: waitingVehicle._id,
});

// Shift queue positions
await WaitingVehicle.updateMany(
{
queuePosition: { $gt: waitingVehicle.queuePosition },
},
{
$inc: { queuePosition: -1 },
}
);
}
```

```
// ================================
// Return Exit Response
// ================================

return res.status(200).json({
success: true,
message: "Vehicle exited successfully",
data: {
vehicleNumber,
slotNumber: slot.slotNumber,
entryTime: parkingRecord.entryTime,
exitTime,
parkingDuration: `${durationInHours} hour(s)`,
parkingFee,
},
});

} catch (error) {
return res.status(500).json({
success: false,
message: error.message,
});
}
};
```

```
const getParkingHistory = async (req, res) => {
try {
const history = await ParkingRecord.find()
.populate("vehicle")
.populate("slot")
.sort({ createdAt: -1 });

res.status(200).json(history);
} catch (error) {
res.status(500).json({
message: error.message,
});
}
};

module.exports = {
vehicleEntry,
vehicleExit,
getParkingHistory,
};
```

Then models

ParkingRecords, slot.js, vehicles.js and waitingVehicles.js

Then routers

DashboardRoutes, slotRoutes, vehiclesRoutes, queueRoutes